

SalesFlow CRM

Customer Registry & Ticketing Management Control Console

PROJECT NATURE

CRM & Support Desk Console

DEVELOPED BY

Gowtham

DATABASE ENGINES

MongoDB & Local JSON Failover

DATE OF REPORT

13 July 2026

1. Introduction

Project Title: SalesFlow CRM (Customer Registry & Support Ticketing Platform)

Project Nature: An enterprise-ready full-stack Customer Relationship Management (CRM) and support ticketing platform built using the MERN stack. The system features a responsive React dashboard, support ticket management desks, workload tracking, satisfaction feedback ratings, global navigation routing, and a hybrid database engine that automatically falls back to local JSON storage if the database server is offline.

Team Members:

- **GOWTHAM** – Lead Full Stack Developer (Frontend, Backend, Database, Diagnostics, Testing & Documentation).

2. Project Overview

Purpose: To deliver an all-in-one operations control console that empowers sales and support teams to register customers, track contract Lifetime Value (LTV), resolve incoming support tickets within SLA target countdowns, balance workloads across active agents, and monitor client feedback reviews.

Key Features:

- **Secure User Registration & Login:** Allows users to register accounts securely and login with credentials.
- **Customer Management (CRUD):** Enables complete customer lifecycle management, including creating, reading, updating, and deleting customer profiles.
- **Support Ticket Management:** Tracks incoming support requests, assigning agents, and monitoring status progress from open to closed.
- **Customer Interaction History:** Logs chronological history of calls, meetings, and emails for each registered client in a detailed timeline.
- **Feedback Management:** Collects customer satisfaction reviews and rating stars to measure service quality.
- **Reminder Management:** Allows setting follow-up actions and reminders for customer calls or support ticket actions.
- **Dashboard Analytics:** Visualizes customer growth statistics and support agent ticket workloads via interactive charts.
- **Search & Filter Customers:** Supports search queries and status filters on customer registries in real-time.
- **JWT Authentication:** Secures API endpoints and manages user sessions utilizing JSON Web Tokens.
- **MongoDB Database Integration:** Stores customer profiles, support tickets, and agent workloads reliably in MongoDB.
- **Responsive User Interface:** Provides a clean, modern dashboard layout that fits screens of all sizes.
- **RESTful API Architecture:** Coordinates data exchange between Vite/React client and Express backend securely.

3. Architecture

The system utilizes a modern multi-tier model-controller layout. State routing is decoupled from database engines through wrapper abstraction models, supporting seamless online/offline failovers.

Frontend (React.js)

Built using React 18 for fast interface rendering. Context providers maintain authentication data across components. Dynamic dashboards are built with `recharts` components, and CSS transitions power slide-out drawers.

- **AuthContext:** Coordinates session logins, registrations, and updates localStorage credentials.
- **ThemeContext:** Manages Light/Dark mode state and saves user display preferences.
- **Layouts:** Includes sidebar panels, search bars, and popup alerts feeds.

Backend (Node.js & Express.js)

Developed using Node.js and Express.js, providing secure REST APIs for routing and authentication. Routes parse incoming JSON payloads, dispatching operations to dedicated controller classes.

Hybrid Dual-Engine Storage Fallback

The database layer operates an intelligent switcher configuration:

1. **Server DB Failover:** MongoDB connects via Mongoose. If connection times out (3000ms), the backend switcher binds model actions to read/write JSON files inside `server/data/*.json`.
2. **Client API Failover:** The axios client handles network errors. If Express port 5001 is offline, operations switch to local browser database cache `localStorage` seamlessly.

4. Setup Instructions

Prerequisites: Node.js (v18+), npm (v9+), Git CLI, and MongoDB (Optional, as the local JSON fallback operates autonomously if MongoDB is unavailable).

Installation Steps

1. Clone the repository:

```
git clone https://github.com/nikhithasree750-ux/CUSTOMER_REGISTRY.git
cd CUSTOMER_REGISTRY
```

2. Install packages for the server:

```
cd server
npm install
```

3. Install packages for the client:

```
cd ../client
npm install
```

Environment Configurations

Create a `.env` file in the `server/` directory:

```
PORT=5001
MONGODB_URI=mongodb://127.0.0.1:27017/customer_registry
NODE_ENV=development
```

5. Folder Structure

The codebase is split into frontend (client) and backend (server) directories:

```
CUSTOMER_REGISTRY/  
├─ client/                                # Frontend Vite + React Project  
│   └─ src/  
│       ├── components/                 # Inputs, Print Profiles, Modal Dialogs  
│       ├── context/                   # AuthContext & ThemeContext  
│       ├── layouts/                   # Sidebar & Header frames  
│       ├── pages/                     # Dashboard, Customers, Tickets, Agents, Feedback, Settings  
│       ├── routes/                   # AppRoutes protected managers  
│       └─ services/                   # api.js client queries & offline fallback  
└─ server/                              # Backend Express.js Server  
    ├── config/                        # db.js switcher configurations  
    ├── controllers/                   # Controller endpoints logic handlers  
    ├── data/                          # JSON local fallback database files  
    ├── models/                        # Database schema models & JSON wrappers  
    └─ routes/                         # REST routing endpoints
```

6. Running the Application

Start both servers concurrently using separate terminal tabs:

Step 1 (Backend): Navigate to `server/` and run `npm start`. Runs on port 5001.

Step 2 (Frontend): Navigate to `client/` and run `npm run dev`. Starts the Vite UI server on port 5174.

7. API Documentation

All endpoints map requests onto port 5001 under the `/api` prefix.

Method	Endpoint	Request Body / Parameters	Response (Success)
POST	<code>/api/users/register</code>	<code>{ name, email, phone, password, role }</code>	<code>{ _id, name, email, phone, role, avatar }</code>
POST	<code>/api/users/login</code>	<code>{ email, password }</code> (Accepts phone as email)	<code>{ _id, name, email, phone, role, avatar }</code>
GET	<code>/api/customers</code>	Params: <code>search, status, sort, page, limit</code>	<code>{ customers: [...], pagination: {...} }</code>
POST	<code>/api/customers</code>	<code>{ name, email, phone, company, status, ltv, notes }</code>	<code>{ _id, name, email, phone, company, status, ltv }</code>
PUT	<code>/api/customers/:id</code>	<code>{ company, status, ltv, notes }</code>	<code>{ _id, name, company, status, ltv }</code>
GET	<code>/api/tickets</code>	Params: <code>status, priority, agentId, search</code>	<code>{ tickets: [...] }</code>
POST	<code>/api/tickets</code>	<code>{ customerId, description, priority }</code>	<code>{ _id, ticketId, description, priority, dueDate }</code>
PUT	<code>/api/tickets/:id</code>	<code>{ status, assignedAgentId, newComment: { author, text } }</code>	<code>{ _id, ticketId, status, comments, timeline }</code>
GET	<code>/api/agents</code>	None	<code>[{ _id, name, specialty, status }]</code>

Method	Endpoint	Request Body / Parameters	Response (Success)
POST	/api/feedbacks	{ customerId, rating, comment }	{ _id, customerId, rating, comment }

8. Authentication & Role Security

Authentication verifies credentials utilizing password comparison. User login queries handle dynamic string format matching, validating *either* registered email handles *or* international phone digits. User roles restrict views:

- **Manager Role:** Elevates user rights to adjust agent details, monitor workloads, configure database servers, and delete client history logs.
- **Agent Role:** Limits users to edit customer directories, comments logs, and update ticket steppers.

9. User Interface

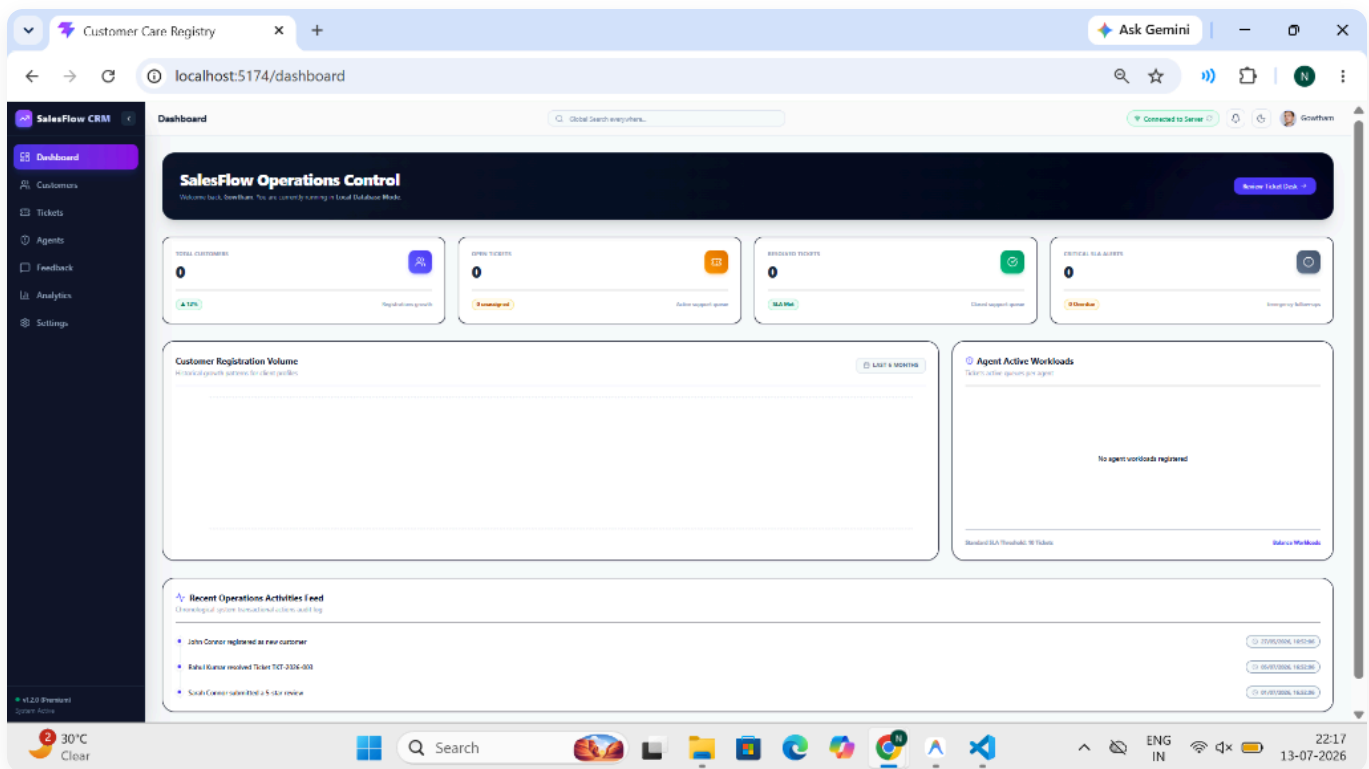
Below are placeholders for the interface screens. To update, copy your captured screenshots from <http://localhost:5174/> and save them in your local directory as indicated below.

9.1 Login Interface Screen

Save screenshot as `images/login.png`

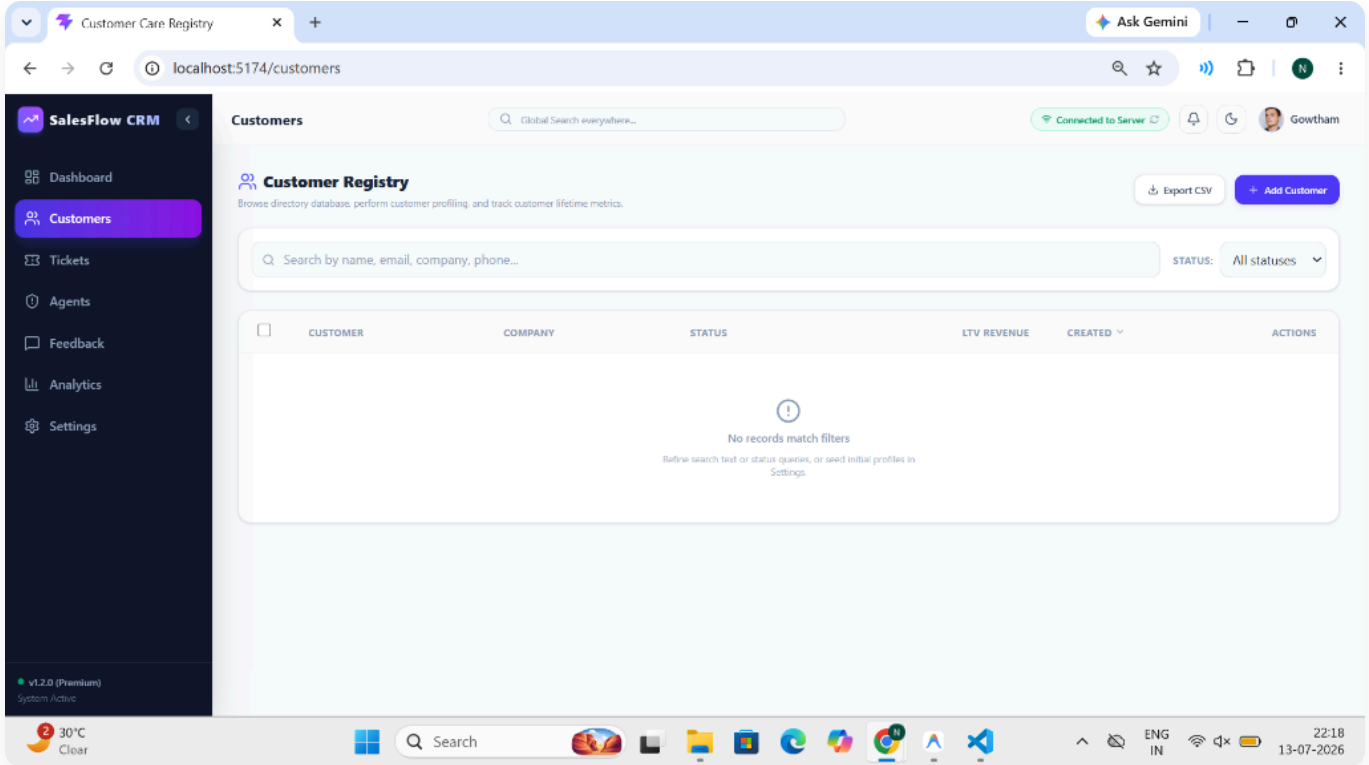
9.2 Operations Dashboard Control Room

Save screenshot as `images/dashboard.png`



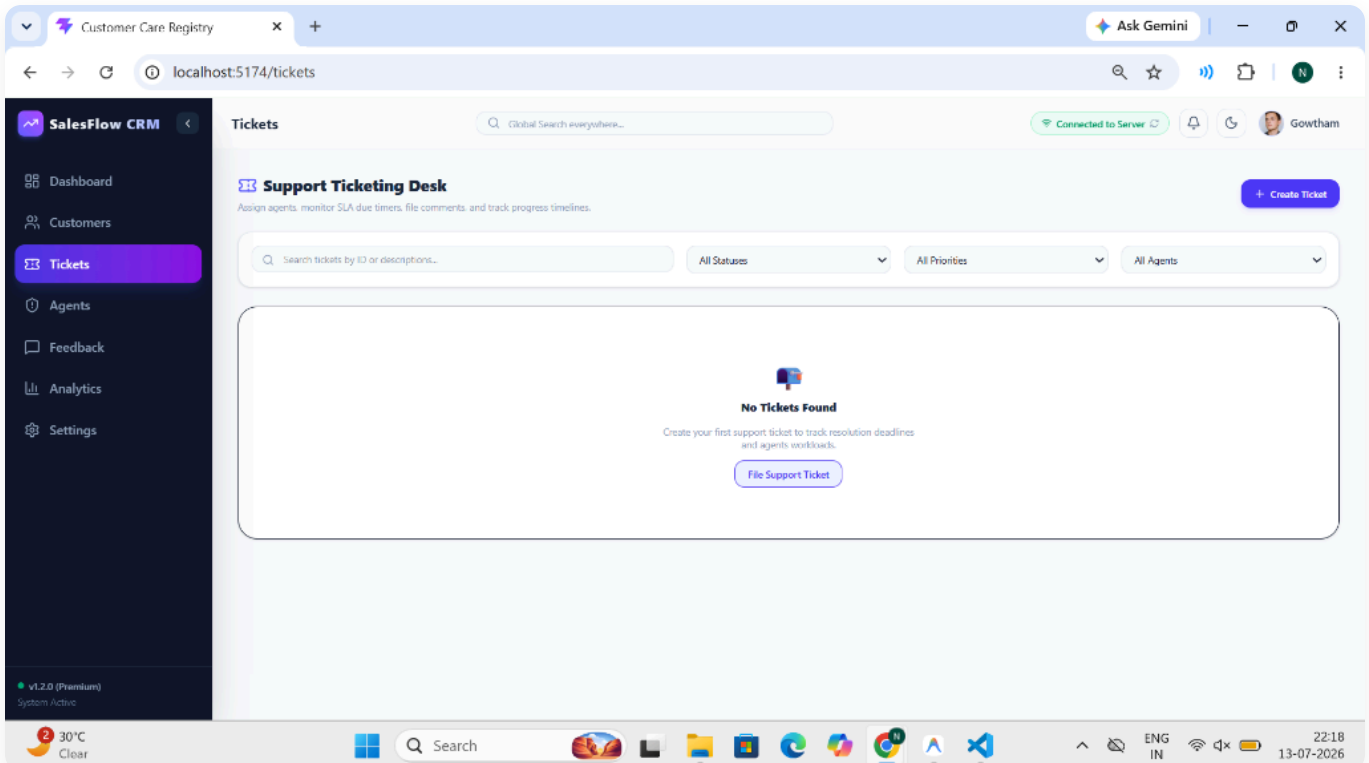
9.3 Customer Registry & Sliding Profile Drawer

Save screenshot as `images/customers.png`



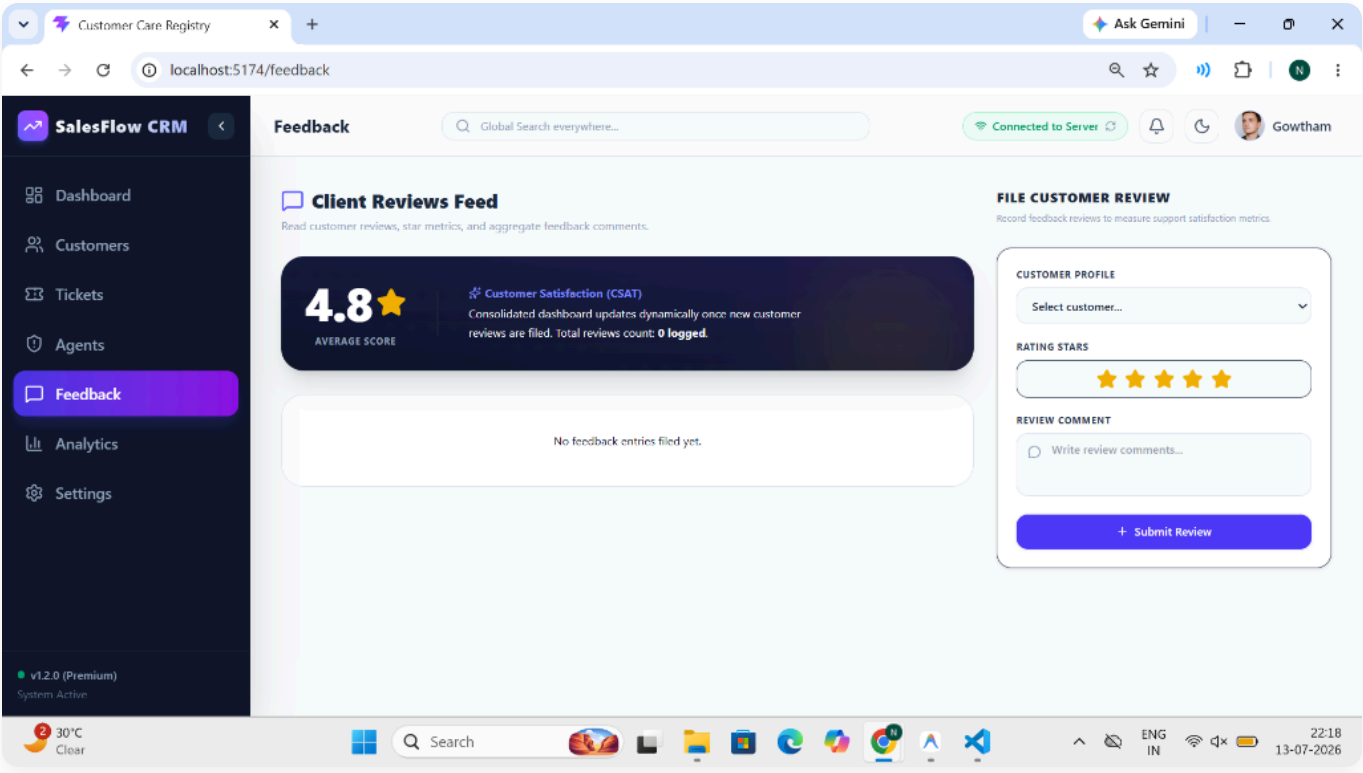
9.4 Support Tickets Desk & Stepper Workspace

Save screenshot as `images/tickets.png`



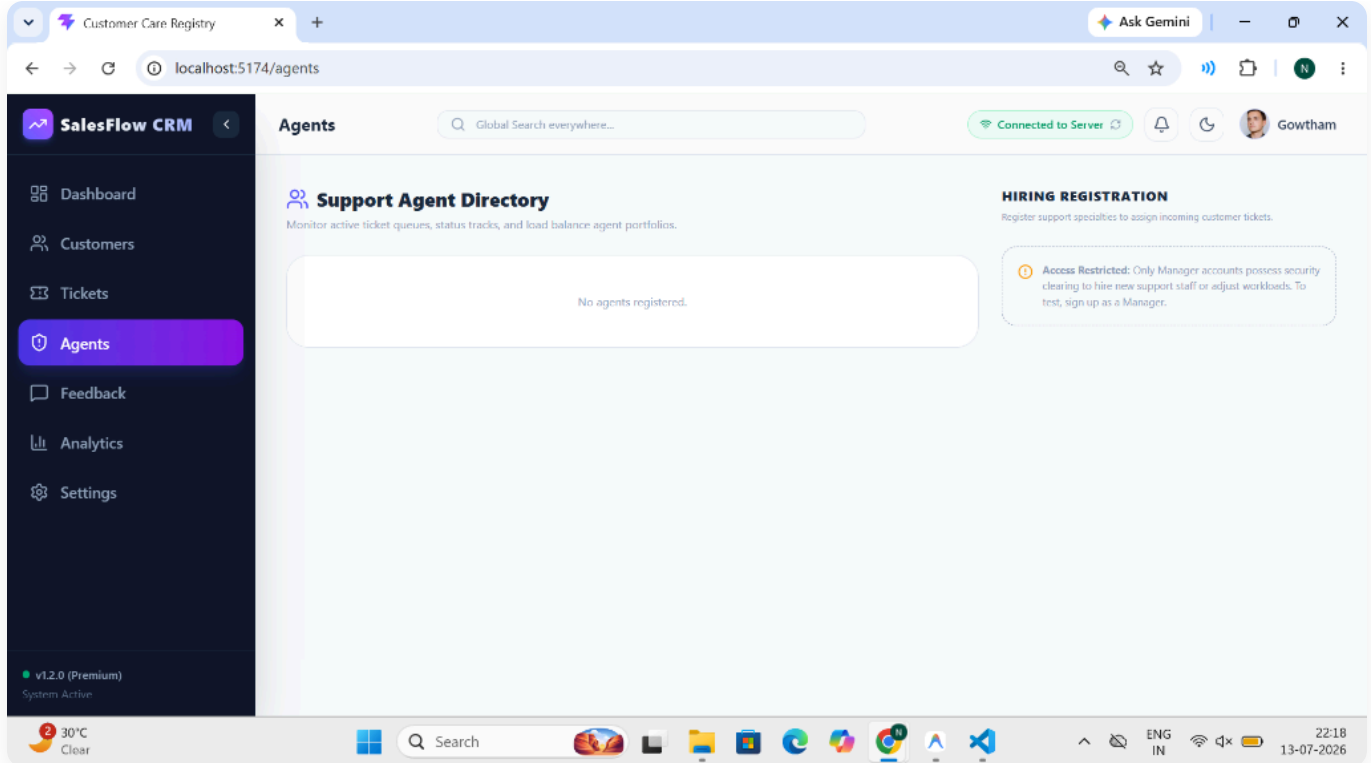
9.5 CSAT Customer Reviews Board

Save screenshot as images/feedbacks.png



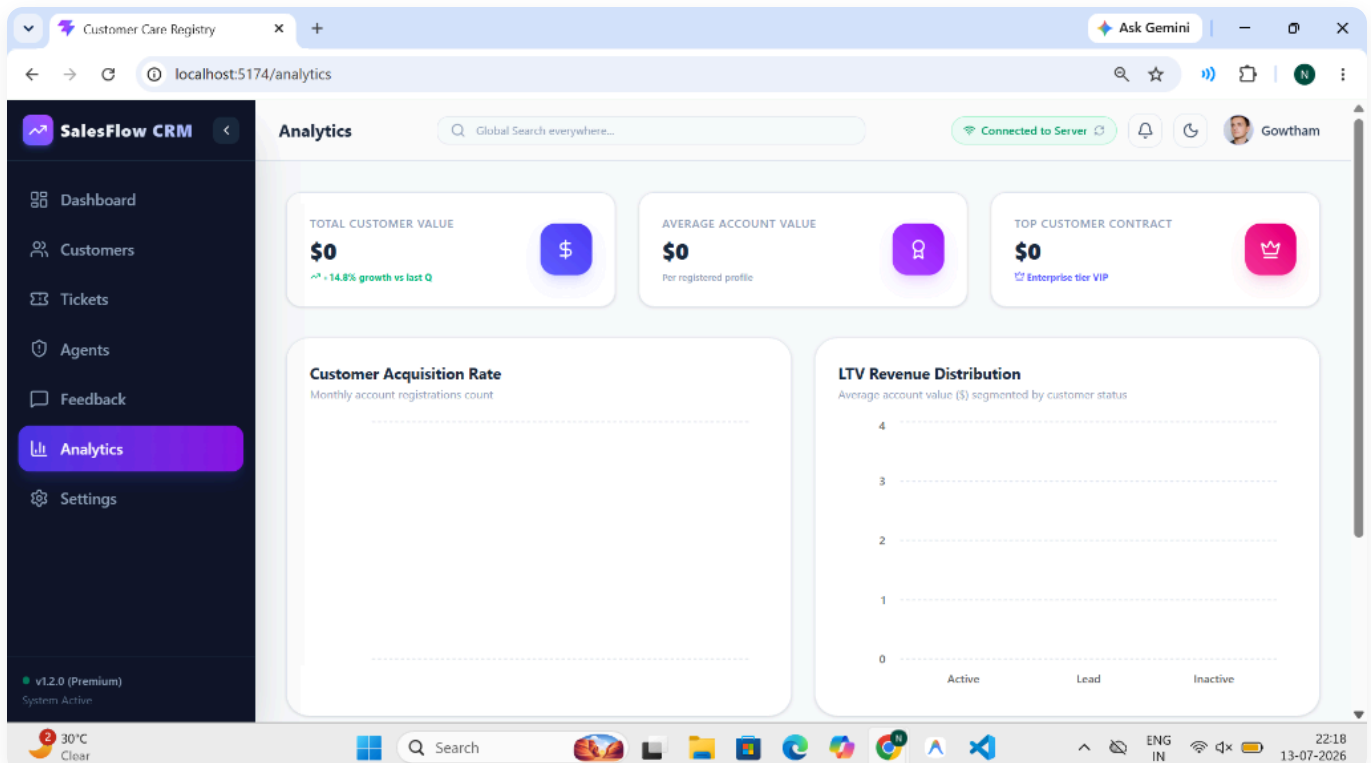
9.6 Support Agent Directory & Hiring Desk

Save screenshot as images/agents.png



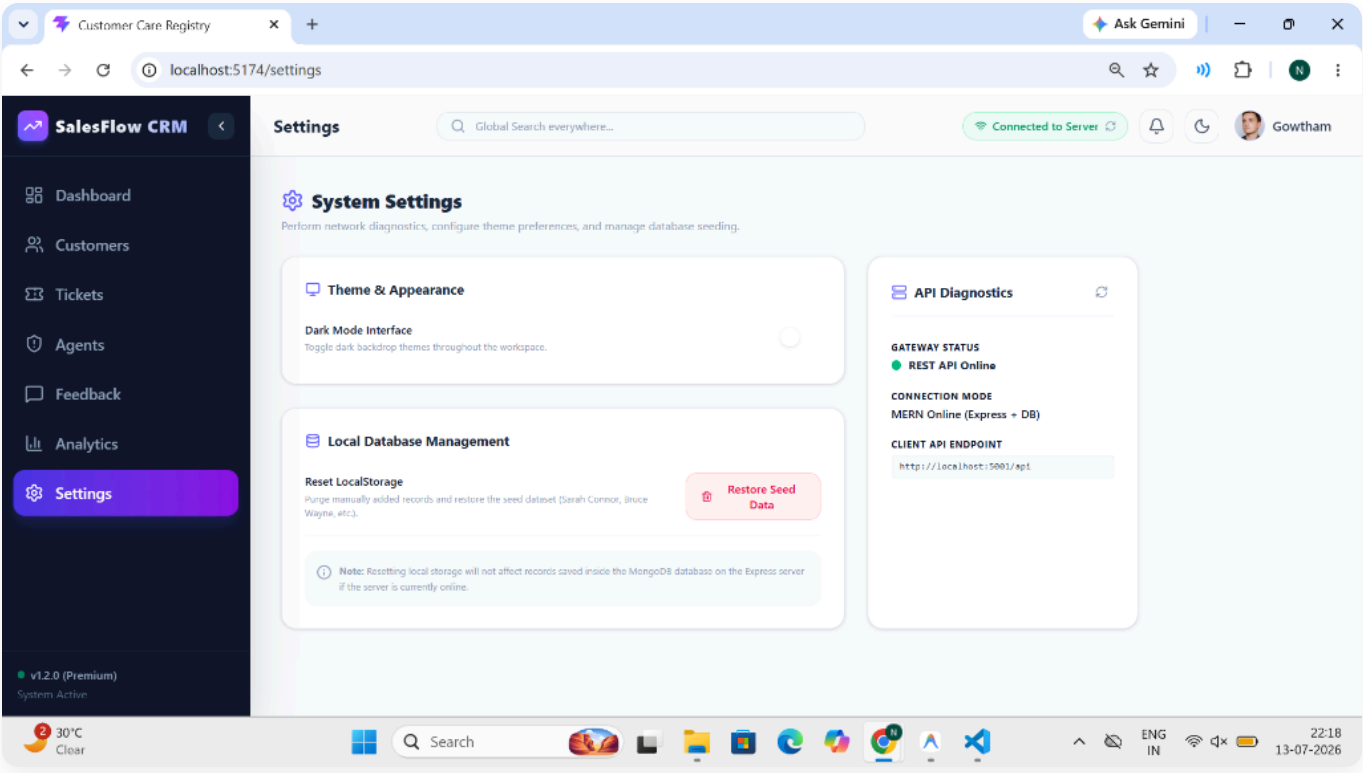
9.7 Advanced Analytics & LTV Reports

Save screenshot as `images/analytics.png`



9.8 Diagnostics & DB Settings Panel

Save screenshot as `images/settings.png`



10. Testing

Testing Strategy: Tests were split into functional inputs check, SLA due-timers check, and offline fallback databases validations.

- **Email/Phone Validations:** Verified correct formatting validations during agent registrations.
- **SLA Calculations:** Checked that `Critical` tickets trigger 4-hour SLA due deadlines and generate alert countdown flags.
- **Database Offline Failover:** Shut down MongoDB service. Express console logged failover and data was loaded from `server/data/customers.json` . Shutdown Express server, client logged fallback database mode and loaded/saved details via `localStorage` .

11. Screenshots or Demo

GitHub Repository: https://github.com/nikhithasree750-ux/CUSTOMER_REGISTRY

Local Deployment URL: <http://localhost:5174/>

12. Known Issues

- **Port Conflict (Windows Port 5000):** Windows services like SSDP Discovery block Port 5000. *Solution:* Server set to bind to Port 5001.
- **Cached Empty LocalStorage:** Old localStorage configurations cause blank dashboard graphs. *Solution:* Navigate to Settings and click "Restore Seed Data".

13. Future Enhancements

- **AI-powered chatbot for customer support:** Integrate natural language processing assistants to resolve common customer queries automatically.
- **Email and SMS notification integration:** Notify clients and agents immediately when tickets are created, assigned, or resolved.

- **Mobile application for Android and iOS:** Develop native mobile apps to manage the CRM and support desk on the go.
- **Cloud deployment using AWS or Azure:** Host the MERN stack application on reliable cloud infrastructure for higher availability and scalability.
- **Advanced analytics and business intelligence dashboard:** Implement deep insights, predictive churn analysis, and revenue forecasting charts.
- **Multi-language support:** Localize the user interface to support multiple languages for global operations.
- **Role-based permission management:** Implement granular access controls for managers, agents, and client accounts.
- **Real-time notifications using WebSockets:** Push updates to support boards and activity feeds instantly without manual refreshes.
- **File attachment support for tickets:** Allow users to upload screenshots and logs directly to support tickets.
- **Automated backup and recovery system:** Ensure database integrity with automatic scheduled backups to cloud storage.